

Helix OTA — Real Test-Coverage Audit (2026-06-23)

Revision: 1 **Last modified:** 2026-06-23T12:30:00Z **Scope:** server (Go control plane) + owned ota-* Go submodules. Challenges/HelixQA + cluster covered in sibling research docs. **Method:** real go test -cover / -race / go vet runs + mutation-thinking per type + **one live mutation proof** on the security-critical OTA trust boundary. NO new tests, NO inflated numbers (§11.4.6). Evidence: the raw .txt files in this directory (§11.4.83).

Measured server coverage (FACT — all PASS, go vet clean, -race clean)

Package	Cover	Package	Cover
internal/health	100.0%	internal/api	89.0% (Gin via httptest)
internal/fabric	96.7%	internal/device	70.0%
internal/transport	94.9%	internal/store	47.9% $\triangle$ understated (pgx)
internal/deviceemu	94.0%	internal/rollout	28.2% $\triangle$ understated (pgx)
internal/config	90.0%	cmd/*, tools/loadtest	0.0% (mains)

Measured submodule coverage (FACT — all PASS)

- ota-artifact-validator 100.0% · ota-rollout-engine 100.0% · ota-telemetry-schema 98.9% · ota-protocol 98.6% — each ships real stress + chaos + (validator/protocol) security tests.
- Android bricks (JVM): ota-android-agent (47 @Test core + 5 on-device), ota-update-engine-bridge (27 @Test core). Coverage % **UNCONFIRMED** — Gradle/JaCoCo not invoked; on-device needs a device.

Anti-bluff headline — MUTATION-PROVEN REAL

Mutated `ota-artifact-validator/stages.go:110` (the S3 ed25519 signature check) to always-accept — the classic bluff. **Result: MUTANT KILLED — 11 test functions FAILED** across unit + security + chaos (incl. `TestChaosNeverAcceptsCorruption` catching 40/120 corrupted artifacts). Restored → green. **The OTA security trust boundary is covered by genuine anti-bluff tests, not bluff.** Proof: `mutation-proof-validator-s3.txt`. No bluff tests were found in the sampled set; mocks are confined to `*_test.go` (§11.4.27 OK).

Honest gaps (FACT — unrun/environment-gated, NOT bluff)

- **G1 (highest):** store 47.9% / rollout 28.2% **understate** real coverage — the uncovered statements are the `pgx/Postgres Repository` (the *production* persistence target, `architecture.md` §4), tested by `//go:build integration` suites that boot a real Postgres via the containers `brick.go vet -tags integration` compiles clean — but the integration suite was **NOT RUN** in this audit (UNCONFIRMED PASS).
- **G2:** shell `e2e (pipeline_signed.sh, 60 assert points)` + security (`security_probes.sh, 59 hard asserts, self-hosts a real server`) suites exist + are real, but aren't chained into one mechanical runner.
- **G3:** Android JVM JaCoCo coverage uncaptured; on-device A/B needs a device.
- **G4:** no HTTP load/scaling test with p50/p95/p99 latency histogram (§11.4.85); `tools/loadtest` exists at 0% cover.
- **G5:** 7 benchmarks exist but no baseline/regression registry; no Go-layer DDoS/fuzz beyond shell injection probes + 2 rate-limit tests.

Prioritized gap plan (risk-descending, §11.4.132)

1. **[HIGHEST]** Run + capture the Postgres integration suite (`-tags integration, podman`) → closes G1, turns the production persistence layer from UNCONFIRMED to captured.
2. **[HIGH]** HTTP load/scaling test with latency histogram (wire `tools/loadtest`) → G4.
3. **[HIGH]** Android JVM JaCoCo + on-device A/B with evidence → G3.
4. **[MED]** Aggregate shell `e2e/security` into one mechanical runner → G2.
5. **[MED]** `benchstat` baseline registry; rate-limit saturation + `go test -fuzz on upload/telemetry parsers` → G5.
6. **[LOW]** smoke tests for `cmd/* mains`.

Bottom line

The Go layer (server + ota-* bricks) has **strong, genuinely anti-bluff coverage** — mutation-proven on the security-critical path, race-clean, vet-clean. The real gaps are **unrun/environment-gated** (Postgres integration, Android, load-latency), not bluff. The only place the headline numbers mislead is the default suite understating Postgres coverage — documented honestly here, not papered over.